Compute Playground | CDEF Computing

Mini project report - deterministic local Python simulation

Objective. Accept tenant-owned computing tasks, select suitable Edge, Fog or Cloud resources, maintain simulated VM records and report placement, resource use, QoS and access-control outcomes. The project runs locally without paid infrastructure.

Scope. Three configurable nodes, 18 baseline tasks, three placement policies, optional deadline queue, protected decision explorer, timeline, seeded workloads and illustrative compute energy, virtual time, create/stop VM operations, tenant ownership checks, CSV exports and matplotlib charts. All numerical results in this report come from the included experiment runner.

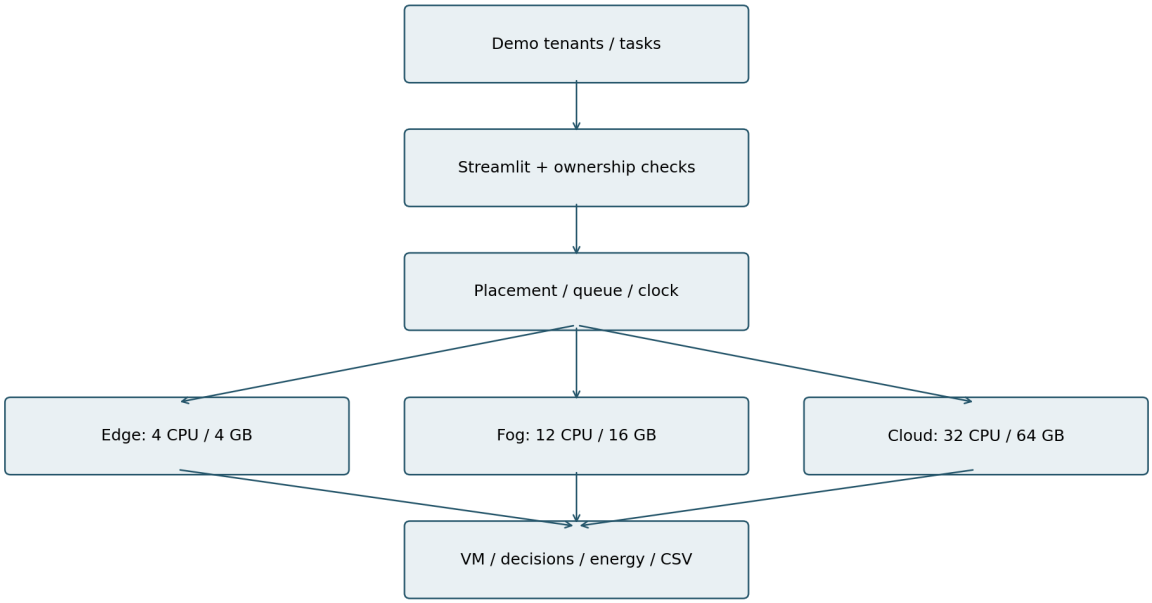
Approach. A Python engine owns state and authorization. Streamlit supplies controls and tables. A separate runner executes the same engine without the interface, making results repeatable and inspectable.

Submission guide. README.md contains setup and demonstration instructions. results/ contains task, VM and utilization CSVs. evidence/ contains experimental setup, expected outcomes, actual outcomes, security and lifecycle records. docs/ contains this report, architecture and slides.

Student details. The team must add its actual names, roll numbers and contribution record in docs/team_details.md before final submission.

Interpretation. These are model predictions rather than physical server benchmarks. CPU units and node speeds are abstract parameters. No hypervisor or cloud account is required.

2. Architecture and data model



Tasks pass through resource and latency admission checks. Placement reserves CPU/RAM and creates one VM per task. The clock processes completion events, releases reservations and records utilization snapshots. Protected record methods check the acting tenant before returning or modifying data.

Node	CPU	RAM GB	Delay ms	Speed
Edge	4	4	5	1
Fog	12	16	25	2
Cloud	32	64	100	4

Task fields: ID, tenant, CPU, RAM, sensitivity, work and arrival time. VM fields: ID, task ID, owner, host, reserved CPU/RAM, creation/stop time and status. Engine records include selected node, response estimate, finish time and decision reason. Duplicate IDs and invalid numeric inputs are rejected.

3. Placement algorithms and QoS model

Processing time in ms = work / (reserved CPU units x node processing speed). Work is CPU-ms at speed 1. Predicted response time = waiting time + processing time + total network delay. CPU/RAM remain allocated through processing; network delay does not reserve compute resources. Completed status denotes compute completion.

Budgets are 60 ms for high sensitivity, 180 ms for medium, and 600 ms for low. Every policy first removes nodes lacking CPU/RAM or exceeding the predicted response budget. With queue off, no feasible node means immediate rejection. With queue on, statically feasible but temporarily blocked work waits, otherwise it is rejected. No migration is modelled. The deadline is arrival plus budget; waiting counts against it.

Lowest latency first. Choose the feasible node with minimum network delay. This does not necessarily minimize total response time because node processing speed differs.

Capacity aware. For each feasible node compute projected utilization U = max(projected CPU fraction, projected RAM fraction). Score = w x (response / budget) + (1-w) x U. Choose the minimum. Weights are .8, .5 and .2 for high, medium and low sensitivity. Node name breaks equal scores deterministically.

Energy aware. Score = .5 x (response / budget) + .5 x (estimated task energy / largest task-energy estimate across nodes). All policies filter the same eligibility constraints. Joules use hypothetical incremental power coefficients, not measurements.

Example: 2 CPU, 1 GB, 80 work, high sensitivity on Edge requires 40 ms compute and 45 ms predicted response, satisfying the 60 ms budget. High-sensitivity tasks cannot use Cloud at its baseline network delay of 100 ms.

The clock is absolute and finite. At equal timestamps, completions free resources before new arrivals. Simultaneous arrivals follow workload list order. A task keeps its reserved CPU rather than sharing it dynamically. These choices make repeated experiments identical.

4. Implementation, VM lifecycle and access control

simulator/engine.py contains Task and Engine. submit validates inputs, advances to arrival time, evaluates candidates, stores a decision and optionally creates a VM. advance processes due completions in timestamp/ID order. stop verifies ownership and cancels running work. run_until_complete resolves all running and queued work. Queued work reserves nothing and has no VM. Resource releases retry earliest deadlines first, then arrival and ID. Unstarted tasks expire at their deadlines.

VM creation is a reservation record, not a guest operating system. A VM starts running and becomes completed or stopped. A manually stopped task becomes cancelled. Release checks current VM status so repeated stopping and later clock advancement cannot release resources twice.

records, vms, get_task, stop, timeline, decisions and cancel_queued enforce tenant ownership. Callers receive copies rather than mutable internal records. Denied operations produce an audit entry. Shared aggregate resource utilization is intentionally public.

The demo identity selector deliberately allows classroom impersonation. It demonstrates authorization boundaries, not authenticated identity. Published experiments contain synthetic cross-tenant data and are not a confidential production datastore. No real passwords or sensitive information are used.

Streamlit stores the engine in session state. Policy and Edge parameter edits take effect on reset. Custom submissions use current virtual time. Sample loading advances through three arrival batches at 0, 50 and 100 ms. Earlier tasks can therefore complete during loading.

Twenty-five automated tests cover queues, deadlines, energy, private timelines/decisions, scheduled-arrival collisions, and resource bounds, completion timing, rejection/fallback, denied view/stop, idempotent stopping, invalid numeric input, duplicate IDs, backward time, policy difference and repeatability. Tests run with the standard unittest module.

5. Experiment design and evidence

Policy comparison. Setup: 18 tasks, same order and arrivals, fresh state for both policies. Expected: Compare allocation, response time and peak utilization. Evidence: baseline_* CSVs.

Overload. Setup: 20 simultaneous tasks requesting 4 CPU and 4 GB each. Expected: Fallback, then reject when no feasible capacity remains. Evidence: overload_* CSVs.

Capacity change. Setup: Increase Edge CPU/RAM from 4/4 to 12/12. Expected: More Edge placements may change latency. Evidence: capacity_increase_* CSVs.

Latency change. Setup: Increase Edge network delay from 5 to 90 ms. Expected: High-sensitivity tasks become ineligible for Edge. Evidence: latency_increase_* CSVs.

Tenant isolation. Setup: Tenant B requests view and stop of Tenant A task. Expected: Both denied with resources unchanged. Evidence: evidence/security.json.

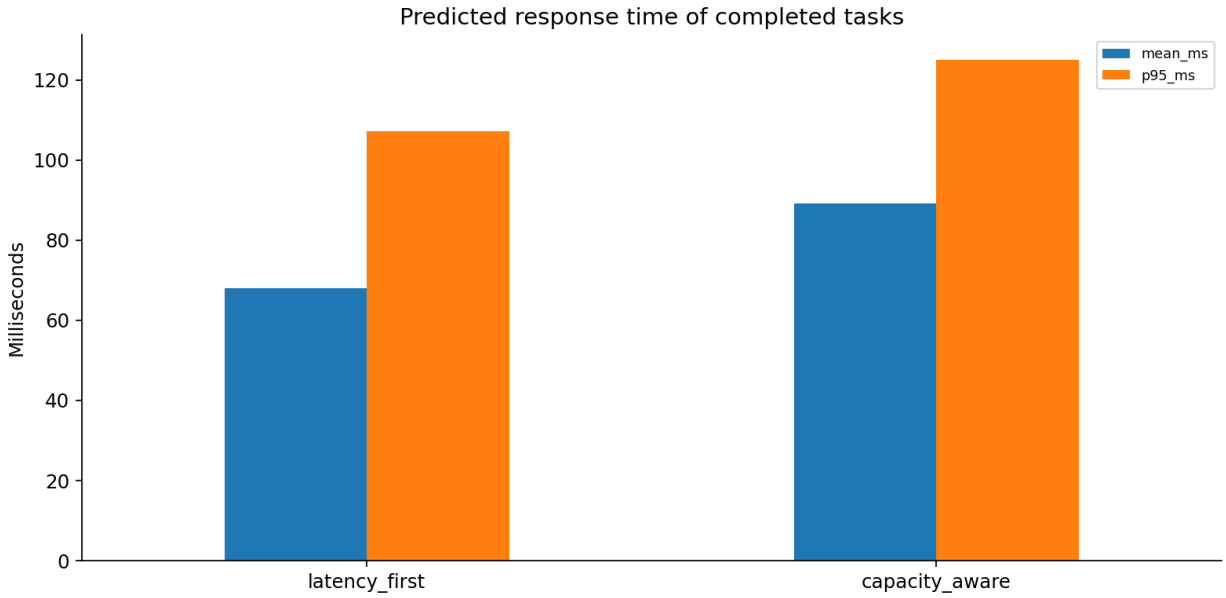
VM lifecycle. Setup: Stop a running VM twice, then advance the clock. Expected: Resources restored once; task cancelled. Evidence: evidence/lifecycle.json.

Original baseline and overload runs keep queues off. The baseline has 18 tasks over two tenants with CPU requirements 1-4 units, RAM 1-4 GB and all sensitivity classes. Three batches preserve overlap and resource contention. Exact inputs are saved in workloads/baseline.json. Every scenario uses a new engine, preventing state contamination.

evidence/experiments.json records actual outcomes alongside expectations. Resource CSVs retain every allocation/release snapshot, including peaks that disappear after all tasks finish. CSVs and charts regenerate with python -m experiments.run_experiments.

6. Measured results

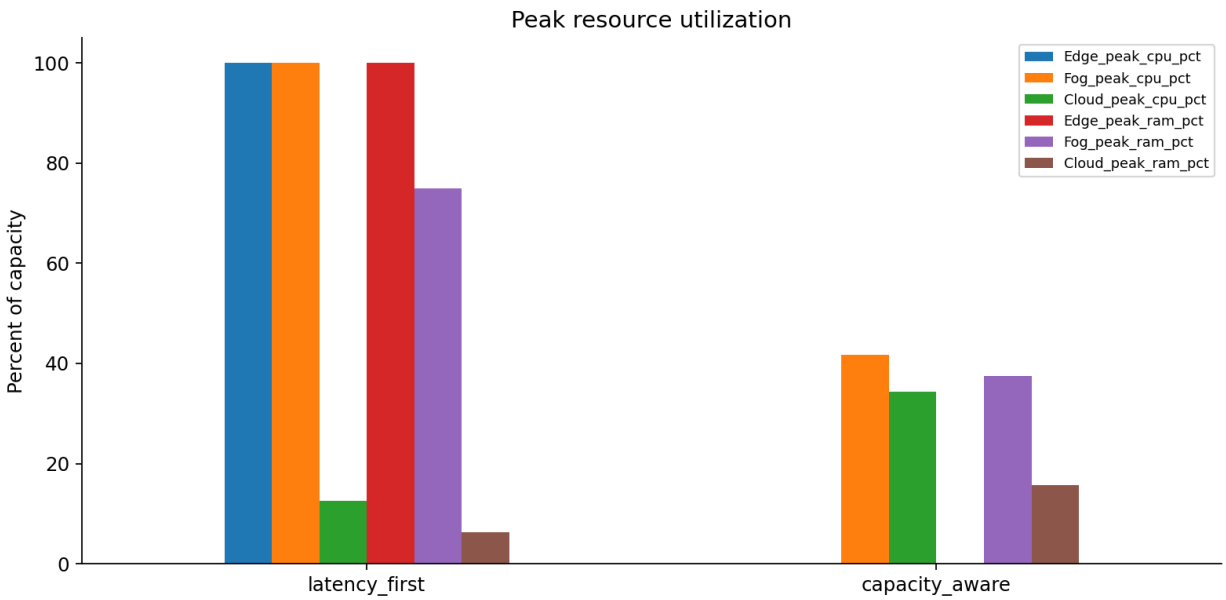
Scenario	Done / total	Rejected	Mean ms	P95 ms
latency	18 / 18	0	68.06	107.25
capacity	18 / 18	0	89.17	125.0
overload	12 / 20	8	110.83	125.0
capacity_increase	18 / 18	0	70.56	105.0
latency_increase	18 / 18	0	76.39	170.0



Both baseline policies completed 18/18 tasks. Capacity-aware increased mean predicted response from 68.06 to 89.17 ms while reducing peak Fog CPU utilization from 100% to 41.67%. It selected 9 Fog and 9 Cloud tasks; latency-first selected 2 Edge, 15 Fog and 1 Cloud task. This is a headroom/latency trade-off, not universal superiority.

Overload completed 12/20 tasks and rejected 8: Edge accepted 1, Fog 3 and Cloud 8. Every accepted task met its predicted budget. Increasing Edge capacity shifted its placements from 2 to 8, but mean response increased to 70.56 ms because more tasks used the slower Edge processor.

7. Utilization, conclusions and limitations



Increasing Edge delay to 90 ms produced mean response 76.39 ms and p95 170 ms. Aggregate placement counts stayed the same in this workload. The absence of a count change is an observed result; it does not mean latency had no QoS effect. Strict admission retained 100% predicted compliance among completed tasks in every scenario.

Read QoS beside rejected counts. Response mean and p95 use completed tasks only; p95 uses linear interpolation. Peak CPU/RAM fractions use event snapshots. There is no measured wall-clock server execution, and no claim of scheduling optimality.

Security evidence shows both unauthorized view and stop denied, with resources unchanged. Lifecycle evidence shows a stopped VM releases its reservation exactly once. Automated checks verify bounds and deterministic execution. These results support the required local simulation and basic tenant separation.

Limitations include synthetic capacities and power coefficients, fixed network delay, no VM boot overhead, no failures, no dynamic contention or migration, and demo identity selection. Queues are optional. Future work could add measured latency, authenticated sessions and real execution adapters. No hardware integration is implemented.

Reproduction: install requirements.txt, run unittest, then run experiments.run_experiments. Open app.py with Streamlit for the live demonstration. Evidence and results are generated from the same engine used by the interface. The assignment brief supplies the project requirements; model parameters and formulas are explicit project assumptions.

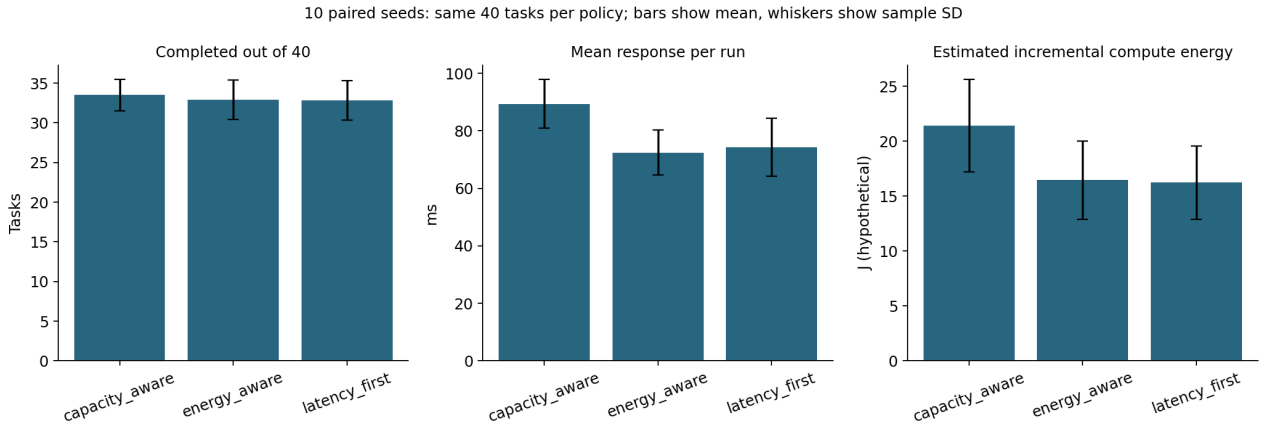
8. Timeline, queues, seeded and energy extensions

Timeline playback advances 20 virtual ms per second and delivers scheduled arrivals. It does not execute real workloads. Tenant-owned event logs and decision snapshots expose node availability, eligibility, projected use, scores and chosen nodes. Queued tasks receive VMs only after allocation. Each retry includes elapsed waiting in its response estimate.

Queue experiment setup: same 14 tasks, fresh latency-first engine, queue off/on. Expected: an additional feasible job can wait, while an urgent job may expire. Actual: queue off completed 12, rejected 2; queue on completed 13, rejected 0 and expired 1. Consumed modeled energy increased from 18.80 to 19.25 J with more completed work. Evidence: results/queue_comparison.csv and queue_on timeline/task/decision files.

Seeded setup: seeds 0-9, 40 tasks each, identical ordered tasks per policy, fresh engine and queues on. Expected: deterministic, paired comparisons with differing policy trade-offs. Actual results below report averages across ten runs. Evidence: results/seeded_runs.csv, seeded_summary.csv and evidence/extensions.json. Whiskers show sample SD, not confidence intervals.

Policy	Done / 40	Mean ms	Wait ms	Energy J
capacity_aware	33.5	89.40	0.033	21.424
energy_aware	32.9	72.48	0.236	16.456
latency_first	32.8	74.30	0.325	16.232



Energy assumption: CPU x hypothetical watts per CPU x elapsed processing ms / 1000. Edge/Fog/Cloud coefficients are 2/6/18 W per CPU. Waiting contributes zero incremental compute energy. Stopped work charges only consumed processing. Idle, memory, network, cooling and boot energy are excluded. Compare joules with completed counts; energy-aware did not achieve the lowest total modeled energy.

Real infrastructure is not plug-and-play. The current VM is a Python record. A future adapter must discover actual capacity, launch and stop workloads, monitor execution, authenticate users, handle credentials and reconcile state after failures. Placement ideas can be reused, but virtual time and predicted work must be adapted. No execution adapter is implemented.